SPEC planning

Wednesday, April 8, 2026 4:34 PM

Requirements

MUST HAVE (MVP)

- Security Gateway
  - Transparent/explicit proxy routing user traffic
  - Request interception before internet access
  - URL reputation check via VirusTotal API
  - File download scanning trigger
  - Allow / Block decision engine
- Authentication
  - Single user login
  - Master password authentication (Passport.js)
- Password Manager (MVP version)
  - Local encrypted password storage
  - Manual retrieval via dashboard
  - No auto-rotation yet
- Monitoring
  - Traffic metadata logging:
    - destination
    - timestamp
    - action
    - request type
- Dashboard
  - Angular UI
  - RxJS streams for live logs
  - Threat history view
- Backend
  - NestJS modular services
  - CRUD entities (≥3 entities with relations)
- Infrastructure
  - Dockerized DB
  - Gateway service containerized
  - Zero Trust session verification

SHOULD HAVE

- Behavioral baseline detection (simple anomaly detection)
- UEBA-lite scoring
- Notification system (Threat notifications)
- Modular services communication
- API key isolation service

COULD HAVE

- Custom session encryption experiment (PIN-based variation)
- Multi-component deployment across VMs
- Simple HIPS simulation / UEBA analytics
- Automated password generation

Recommended Zero Trust Implementation

1. Continuous Session Verification
   Each request to backend checks:
   - authenticated session
   - request origin
   - session age
   - behavioral score (later feature)
2. Risk-Based Access
   Password manager access requires: valid login session AND recent authentication (< X minutes)
3. Device Trust
   Gateway remembers device fingerprint (IP, browser agent, session token) and if it changes it flags anomaly.

System applies Zero Trust by:

- No direct internet access — only gateway allowed
- Every request validated by proxy
- Sensitive actions require fresh authentication
- Behavioral monitoring influences trust
- No implicit trust based on login alone

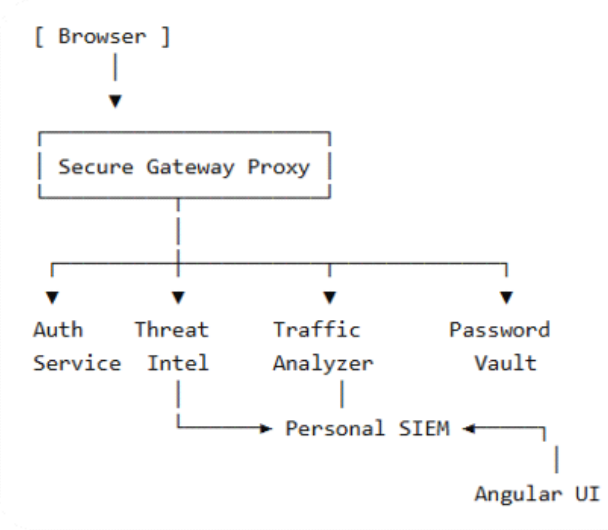
High-Level Architecture

System architecture

Personal Secure Web Gateway Platform - A single gateway VM on which modular security services communicating internally.
High-level components:

- Secure Gateway Proxy
  Intercepts ALL browser traffic, calls security services before forwarding requests
  NestJS, Node HTTP proxy libraries
- Threat Intelligence Service
  Handles external scans - VirusTotal URL check, API keys protected
- Authentication & Zero Trust Service
  Handles login, session validation, re-auth checks, key derivation from master password (master password -> argon2 -> user encryption key)
- Password Vault Service
  Local encrypted credential storage, CRUD passwords, database encrypted using derived key
- Traffic Analyzer (Metadata Collector)
  Collects destination domain, timestamps, action, request type (later user for UEBA)
- Personal SIEM Service
  Receives logs from ALL components, normalize, store events, provide dashboard stream (Angular subscribes via RxJS)
- Angular Dashboard
  User interface: live traffic monitor, threats history, password vault UI, system status
  RxJS used for live log streaming, filtering, reactive updates

Logical Architecture Diagram (Textual)

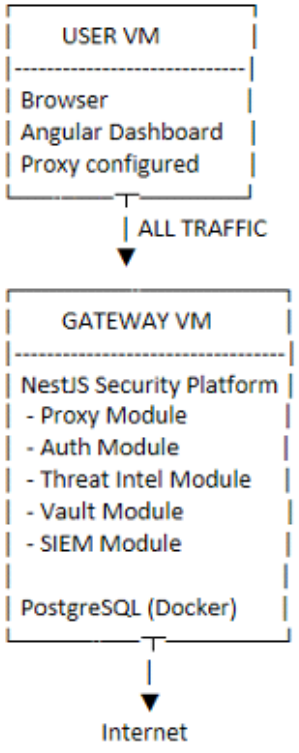


Docker Layout (Gateway VM)

```
gateway-vm
|
├── proxy-service (NestJS)
├── auth-service (NestJS module)
├── threat-service
├── vault-service
├── siem-service
├── postgres-db (container)
└── angular-ui
```

Network Flow & Proxy Algorithm

Vmware Network Topology



Request lifecycle

1. Browser sends request → Gateway Proxy
2. Proxy pauses forwarding
3. URL sent to Threat Service
4. Threat Service queries VirusTotal API
5. Decision returned (SAFE / SUSPICIOUS)
6. Metadata logged to SIEM
7. If SAFE → request forwarded
8. Response returned to user

Security decision

1. Incoming request
2. Zero trust check
3. URL reputation scan
4. Policy engine:
   Block (return warning page) / Allow (forward request)

Zero trust verification

For each request check:

- Valid session - Passport JWT
- Session freshness - timestamp check
- Device consistency - IP + User Agent
- Access scope - role check?

Proxy algorithm

```
async handleRequest(req, res) {

  // 1. Zero Trust validation
  await authService.verifySession(req);

  // 2. Extract destination
  const url = extractUrl(req);

  // 3. Threat intelligence check
  const verdict = await threatService.scanUrl(url);

  // 4. Log metadata
  await siemService.log({
    url,
    user,
    verdict,
    timestamp
  });

  // 5. Policy decision
  if (verdict === 'malicious') {
    return blockResponse(res);
  }

  // 6. Forward request
  proxy.forward(req, res);
}
```

Simulated file download scan

```
IF response header contains:
   content-disposition: attachment
   OR file extension detected
THEN:
   simulate scan event
   call VirusTotal metadata API
   log result
```

Internal Module Communication

Proxy module calls Threat module which emits event to SIEM module.
NestJS EventEmitter: eventEmitter.emit('threat.checked', data);

Logging model

SIEM produces event stream:
RequestAllowed, RequestBlocked, ScanCompleted, LoginEvent, VaultAccess
Angular subscribes: WebSocket → RxJS Observable → UI

Database Schema and Data Model

Database - PostgreSQL (Docker container)

Entities
Main:

- User
- Session
- VaultCredential
- TrafficLog
- SecurityEvent

Encryption flow:
Master password -> Argon2 KDF
-> Derived Vault Key -> Decrypt credentials

Entity relationship:
User - Sessions (1:N)
User - VaultCredential (1:N)
User - TrafficLog (1:N)
User - SecurityEvent (1:N)

Table: `users`

Field	Type	Notes
id	UUID	PK
username	string	unique
password_hash	string	Argon2 hash
created_at	timestamp	
last_auth_at	timestamp	zero trust recheck

Table: `sessions`

Field	Type	Notes
id	UUID	PK
user_id	FK → users	
jwt_id	string	token reference
ip_address	string	device verification
user_agent	string	fingerprint
created_at	timestamp	
expires_at	timestamp	
last_verified_at	timestamp	re-auth logic

Table: `vault_credentials`

Field	Type	Notes
id	UUID	PK
user_id	FK	
service_name	string	github, gmail
username	string	login name
encrypted_password	text	AES encrypted
iv	text	encryption IV
created_at	timestamp	
updated_at	timestamp	

Table: `traffic_logs`

Field	Type	Notes
id	UUID	PK
user_id	FK	
url	text	destination
method	string	GET/POST
verdict	enum	allow/block
risk_score	int	API result
timestamp	timestamp	

Table: `security_events`

Field	Type	Notes
id	UUID	PK
type	string	LOGIN, BLOCK, SCAN
severity	enum	low/med/high
message	text	
metadata	JSONB	flexible
created_at	timestamp	

Cryptography & Security Model

Authentication

When user logs in: Master Password -> Argon2id hash verification -> JWT session issued
Stored in DB: password_hash = Argon2(masterPassword)
Parameters:
    memoryCost: 19456
    timeCost: 2
    parallelism: 1

When user authenticates: Master Password + User Salt (stored) -> Argon2 KDF -> Vault Encryption Key (32bits)

Password encryption (AES-256-GCM)
Each credential encrypted individually. vaultKey -> AES -> ciphertext + authTag + IV
Node example: crypto.createCipheriv('aes-256-gcm', key, iv)

Zero trust
NestJS Guard checks: Request arrives -> (JWT valid? -> Session exists? -> Device matches? -> Last verification recent?)
if not return REAUTH_REQUIRED (frontend asks password again)
Example for sensitive actions (vault/settings/export):
    if now - last_auth_at > 10 minutes
      require password confirmation

User - Gateway communication

Custom encryption between User VM and Gateway VM using HTTPS (self-signed local CA).
Gateway generates local-root-ca.crt and it should be installed on user VM for trusted connection.

API Key Protection (for threat intelligence APIs)
VirusTotal key must never leak, only module accesses key, Angular never sees API key.
Threat module -> environment variable -> Docket secret (.env)

Inside NestJS is used event-driven communication, not direct DB sharing (Proxy emits event, SIEM listens).?

Memory security
Vault key stored: request context memory only
Destroyed when: logout, session expiry, server restart
Implementation idea: In-memory Map<sessionID, derivatedKey>